

Custom SQL Journey Documentation

Awwab Wadekar

June 24, 2026

1 Introduction

I was bored in my vacation, so I thought why not learn how SQL works under the hood with an attempt on a custom implementation?

This is torture. It was not fun.

Anyways the journey is a 5 phase mess:

Phase 1. The OS background

Phase 2. The Volcano Model

Phase 3. The System Catalog

Phase 4. The Parser Dilemma

Phase 5. The Editor Interface

2 Phase 1: The OS background

2.1 Day 1: Serialisation and Deserialisation

Data is often stored in Raw Bytes in Memory. So when we fetch it we actually can't use something like `struct.id` or `struct.name` to access fields.

Instead we use an offset variable to keep track of what we have accessed.

```
struct MyStruct{
    char name[30];
    int id;
}
```

Now here the total memory is $30 * 1 + 4 = 34$ bytes, as one char is 1 byte and one int is 4 bytes.

But usually the OS assigns Memory in terms of Pages, which here are considered to be 4 bytes in size. So you have Page 1...Page 2...Page 3.....Page 8...Page 9. Basically $9 * 4 = 36$ bytes.

Wait a minute, what ? I'm not a Mathematics Graduate but surely $36 > 34$.

So what actually happened here is padding, in the 8th Page, 2 bytes are used for the last 2 elements of the char array.

But we cant exactly put 2 bytes of an int in one page and 2 in another. And we for sure cannot have half a page. So the OS simply pads those 2 bytes up.

Well how do we fix this? The solution : Interpret everything as raw bytes and track offset of every field. Something along the lines of :

```
vector<uint8_t> buffer;
size_t offset = 0;
MyStruct obj;
obj.name = "Awwab";
obj.id = 77;
std::memcpy(buffer.data() + offset, &obj.name,
    sizeof(obj.name))
offset += sizeof(obj.name);
std::memcpy(buffer.data() + offset, &obj.id,
    sizeof(obj.id))
```

```
offset += sizeof(obj.id);
```

Similarly, Deserialization does the same process, just this time the source and destination are swapped.

```
vector<uint8_t> buffer;  
size_t offset = 0;  
MyStruct obj;  
std::memcpy(&obj.name, buffer.data() + offset,  
            sizeof(obj.name))  
offset += sizeof(obj.name);  
std::memcpy(&obj.id, buffer.data() + offset,  
            sizeof(obj.id))  
offset += sizeof(obj.id);
```

2.2 Day 2: First Prototype of a Page

A Page in reference to OS, is a kind of unit of storage. Usually when the computer wants to play around with data, it usually does so in the form of Pages.

Throughout this project we will see my page design get updated repeatedly. For now though, our Page is a somewhat simple class as given as below:

```
// struct for acting as a record
struct [[gnu::packed]] UserRecord
{
    uint32_t id;
    char username[32];
    uint32_t age;
}; // [[gnu::packed]] Prevents compiler padding

class Page
{
private:
    // Exactly 4096 bytes matching typical OS page
    // size
    std::array<uint8_t, 4096> data_buffer;

    // Helper to calculate how large a single
    // record is
    const size_t RECORD_SIZE = sizeof(uint32_t) +
        32 + sizeof(uint32_t); // 40 bytes
    const size_t HEADER_SIZE = sizeof(uint32_t);
        // 4 bytes for
        // storing record count

public:
    Page()
    {
        data_buffer.fill(0); // Zero out the page
        // initially
    }
    const uint8_t* ReadRawData() const { return
        data_buffer.data(); }
    uint8_t* WriteRawData() { return data_buffer.
        data(); }
    // Returns how many records are currently
    // stored (read from the header)
```

```

uint32_t GetRecordCount() const; // returns
    number of records currently in Page

// Appends a record to the page if there is
    space
bool AppendRecord(const UserRecord &record);

// Retrieves a record by its slot index inside
    this page
UserRecord GetRecord(uint32_t index) const;
};

```

Although, we will change much functionality of this class in the future, for now it is a good starting point. We have a clear structure for how data is organized within a page, and the methods for reading and writing records are well-defined.

2.3 Day 3 : Connecting to the Disk

As you might have learnt, data is stored in devices such as magnetic disks or hard drives. On a higher level, data is stored in files. We shall treat one file as basically disk. Much like how the read/write head moves across sectors in a disk, our disk manager moves its pointer across the file with the help of `seekg` and `seekp`.

A high level view is given below.

```
class DiskManager
{
private:
    std::fstream db_file;
    std::string filename;

public:
    DiskManager(const std::string &db_filename) :
        filename(db_filename); // open file
    ~DiskManager(); // close file

    void WritePage(uint32_t page_id, const Page &
        page); // write to a page

    void ReadPage(uint32_t page_id, Page &out_page
        ); // read a page
};
```

Obviously you will notice that we don't have a truly complete logic yet. Mainly we aren't tracking the number of pages, nor is there a clear way defined for the bounds of the file.

This will be addressed in Day 4

2.4 Day 4 : Refining the Disk Manager

After looking at the shortcomings of our simple implementation of a disk manager yesterday, we will extend our definition by 2 new helper functions.

```
// Returns the total number of 4KB blocks
// currently inside the file
uint32_t GetTotalPages();
// Grows the file by one blank page and returns
// its new Page ID
uint32_t AllocatePage();
```

This definition will easily suffice for our needs for much of the journey. Much like a physical read/write head, it smoothly scans through our pages, reads the necessary data and writes at the correct locations. Another benefit of this simplistic generic implementation is that we can often change the Page definition multiple times (as we will soon see in the later phases), we hardly have to change the logic of our disk manager. This is the power of Abstraction at play !

2.5 Day 5 : First Query

Although we obviously need to program a looootttt more code for solving any general query in the domain and rules clearly set by our language, but right now, we just want to show how we can use the current data structures for a specific query.

Right now our data structure stores records with 3 fields : username, id and age. We will perform a query equivalent to : Select username, age FROM ... WHERE age $\geq$

The logic is basically to scan every page in our disk, and then scans every record in a page. This type of sequential scan is a precursor to what is about to come :D.

```
void ScanAndPrintUsersOlderThan(DiskManager &disk,
    uint32_t target_age)
{
    uint32_t total_pages = disk.GetTotalPages();
    // YOUR CODE HERE:
    // 1. Loop through every page_id from 0 to
        total_pages - 1
    for (uint32_t i = 0; i < total_pages; i++)
    {
        // 2. Inside the loop, declare a local
            stack Page instance
        Page local_page;
        // 3. Call disk.ReadPage(page_id,
            local_page)
        disk.ReadPage(i, local_page);
        // 4. Get the record count for that page
        uint32_t record_count = local_page.
            GetRecordCount();
        // 5. Loop through every record index in
            that page using local_page.GetRecord(i)
        for (uint32_t j = 0; j < record_count; j
            ++)
        {
            UserRecord record = local_page.
                GetRecord(j);
            // 6. If record.age > target_age,
                print it to the console!
            if (record.age > target_age)
                std::cout << "User_" << record.
                    username << "_with_Age:_ " <<
                        record.age << std::endl;
        }
    }
}
```

```
}
```

The above code is our function used to calculate the required result of the query. A more general approach that we will see later, will involve using a template or a sort of virtual class being used as a base, on top of which we can build complex expressions that can cover logical questions such as 'is age $\geq$ x ?' and perhaps even mathematical evaluations.

2.6 Day 6 : Buffer Pool Manager

Now, imagine everytime we want some specific data , we have to go to the disk every single time. That would be highly resource consuming, and is usually not preferred when we are dealing with huge loads of data. So we need some sort of "buffer" to store frequently accessed data.

If our disk manager handles the structural "ROM" of our logic, then the buffer pool manager handles the structural "RAM". The logic is basically :

1. Check if Page exists in Buffer.
2. If yes, return it and push page to front.
3. Else first check buffer size. If it is at limit, evict the least recently used Page in the buffer.
4. After at least one free space is in the buffer, pull the page from disk and return it.

A compact view of the class is given below.

```
class BufferPoolManager
{
private:
    static constexpr size_t POOL_SIZE = 3; // Our
        RAM can only hold 3 pages at once

    DiskManager &disk_manager;
    std::array<Page, POOL_SIZE> pool; // The
        physical slots (frames) in RAM
    size_t next_free_frame = 0; // Tracks
        where to insert on a cache miss
    // Inside BufferPoolManager:
    // 1. The list stores page_ids. Front is MRU (
        Most Recently Used), Back is LRU.
    std::list<uint32_t> lru_list;

    // 2. The map stores the frame and the
        iterator (the "bookmark")
    struct PageMetadata
    {
        uint32_t frame_id;
        std::list<uint32_t>::iterator
            list_iterator;
        bool is_dirty;
    };
};
```

```

        std::unordered_map<uint32_t, PageMetadata>
            page_table;
        // Now maps page_id to {frame_id,
            iterator_to_list_node}
public:
    uint32_t cache_hits = 0;
    uint32_t cache_misses = 0;

    BufferPoolManager(DiskManager &disk) :
        disk_manager(disk) {}

    void MarkDirty(uint32_t page_id); \\ mark a
        page as dirty to ensure data consistency.
    Page *FetchPage(uint32_t page_id); \\ Page
        ftching logic as given above
};

```

2.7 Day 7 : Chapter's close

For a final test of our logic built so far, we will test writing to disk, using our buffer pool manager for optimal fetching, test our LRU logic, and ensure that changes written to a page on the buffer pool manager are written back to disk.

Thus our final "test" as such is given below.

```
std::string filename = "integration_test.db";
std::remove(filename.c_str());

// 1. Setup: Load a page
{
    DiskManager disk(filename);
    uint32_t p0 = disk.AllocatePage();
    Page p0_data;
    p0_data.AppendRecord(UserRecord{1, "
        InitialData", 20});
    disk.WritePage(p0, p0_data);
}

// 2. Perform operations with Dirty Bit
{
    DiskManager disk(filename);
    BufferPoolManager bpm(disk);

    // Fetch
    Page *p = bpm.FetchPage(0);

    // Modify the page in RAM
    UserRecord updated{1, "UpdatedData", 99};
    p->AppendRecord(updated); // Assume you added
        an Overwrite/Update method to Page
    // CRITICAL: Mark as dirty!
    bpm.MarkDirty(0);

    // Trigger eviction by filling the pool with
        other pages
    bpm.FetchPage(1);
    bpm.FetchPage(2);
    bpm.FetchPage(3); // Page 0 should now be
        evicted and written to disk
}

// 3. Verify Persistence
{
```

```

    DiskManager disk(filename);
    Page p0_final;
    disk.ReadPage(0, p0_final);

    // If the dirty bit worked, this will be 99,
    // not 20!
    assert(p0_final.GetRecord(1).age == 99);
    std::cout << "Dirty_bit_persistence_test_
        passed!" << std::endl;
}

std::remove(filename.c_str());

```

With that our first phase is officially complete :D. Now our next phase will deal with making our current logic more abstract and generalised. We will also need to ensure appropriate logic for maintaining types for our custom SQL language. To do this we will be making a sort of "volcano like model" whose base is our current logic of fetching pages from the buffer / disk. The upper layers will have the responsibility to be able to perform the necessary complex operations on any data that is passed from the base, according to a set of rules, that are designed to always be expandable.

3 Phase 2: The Volcano Model

3.1 Day 1: Schemas

Ahh, one of the most central parts of SQL is the schema. It lets you define your data in a structured manner. To start off phase 2, we define the Schema class, the Column struct and the TypeID enum class. We begin with our types being only 2 : INT32 and VARCHAR. A column is used to store the type, name length and offset within the schema as metadata. A schema is a collection of columns, that basically defines a table for us. Below is given the skeleton of our new code.

```
// Use an enum for performance-critical type
// checking
enum class TypeId {
    INT32,
    VARCHAR
};

struct Column {
    std::string name;
    TypeId type;
    uint32_t length; // Size in bytes (e.g., 4 for
                     // INT32, N for VARCHAR)
    uint32_t offset; // Pre-calculated byte offset
                     // within a tuple, represents the size of all
                     // columns before it
};

class Schema {
private:
    std::vector<Column> columns;
    uint32_t tuple_size = 0; // Total size of a
                             // single row in bytes

public:
    // Explicitly define a schema by passing a
    // list of columns
    Schema(const std::vector<Column>&
           input_columns) : columns(input_columns);
```

```

~Schema() = default;

const std::vector<Column>& GetColumns(); //
    returns columns as a read only value

uint32_t GetTupleSize(); // returns the size
    of the tuple

uint32_t GetColumnCount(); // returns total
    number of columns

const Column& GetColumn(uint32_t col_idx); //
    returns a specific column
};

```

3.2 Day 2 : Terrific Tuples

When we will be fetching data from our Pages , now we cannot pass them as raw bytes for the higher level executors. Instead we actually need a sort of generalised structure that works for any specific type. Enter the tuple class : a model that allows us to fetch data from any Page, check the appropriate type and return the raw data in that type. We also have a struct called as RID, which we use to uniquely identify the location of our Page and the slot of our record. Here, the slot value will be more relevant in the next phase when we update some of our code in phase 1. A basic idea of today's work is as follows :

```
// Uniquely identifies a physical record on disk
struct RID
{
    uint32_t page_id;
    uint32_t slot_num;

    RID() : page_id(0), slot_num(0) {}
    RID(uint32_t page_id, uint32_t slot_num) :
        page_id(page_id), slot_num(slot_num) {}
};

class Tuple
{
private:
    const uint8_t *data_ptr_; // Zero-copy pointer
                             // to the buffer pool page
    uint32_t size_;           // Size of the tuple
                             // in bytes
    RID rid_;                 // Physical location
                             // of this tuple

public:
    Tuple() : data_ptr_(nullptr), size_(0) {}

    // Initialize the tuple with a pointer to the
    // raw page data
    Tuple(const uint8_t *data_ptr, uint32_t size,
          RID rid)
        : data_ptr_(data_ptr), size_(size), rid_(
            rid) {}

    ~Tuple() = default;
```

```

RID GetRID(); // returns rid
const uint8_t *GetData(); // returns ptr to
    start of Page

// Extracts an integer field using the Schema's
    pre-calculated offset
int32_t GetInt32(const Schema &schema,
    uint32_t col_idx);
// Extracts a string field using the Schema's
    pre-calculated offset
std::string GetVarchar(const Schema &schema,
    uint32_t col_idx);
};

```